Emitting messages at compile time

Document #:	P2758R1
Date:	2023-12-09
Project:	Programming Language C++
Audience:	EWG

Reply-to:	Barry Revzin <barry.revzin@gmail.com>

Contents

§ 1 Revision History

[P2758R0] and [P2741R0] were published at the same time and had a lot of overlap. Since then, [P2741R3] was adopted. As such, this paper no longer needs to propose the same thing. That part of the paper has been removed. This revision now only adds library functions that emit messages at compile time.

§ 2 Introduction

Currently, our ability to provide diagnostics to users is pretty limited. There are two kinds of errors I want to talk about here: static assertions and forced constant evaluation failures.

§ 2.1 `static_assert`

We can use `static_assert`, but the only message we can provide is a string literal. This is useful (better than nothing), but is frequently insufficient. Consider:

```
template <typename T>
void foo(T t) {
    static_assert(sizeof(T) == 8, "All types must have size 8");
    // ...
}
```

What happens when I try to call `foo('c')`? These are the error messages I get:

— MSVC:

```
<source>(6): error C2338: static_assert failed: 'All types must have size 8'
<source>(11): note: see reference to function template instantiation 'void
        foo<char>(T)' being compiled
        with
        [
            T=char
        ]
```

— GCC:

```
<source>: In instantiation of 'void foo(T) [with T = char]':
<source>:11:8:   required from here
<source>:6:29: error: static assertion failed: All types must have size 8
    6 |     static_assert(sizeof(T) == 8, "All types must have size 8");
      |                   ~~~~~~~~~~^~~~
<source>:6:29: note: the comparison reduces to '(1 == 8)'
```

— Clang:

```
<source>:6:5: error: static assertion failed due to requirement 'sizeof(char) ==
        8': All types must have size 8
    static_assert(sizeof(T) == 8, "All types must have size 8");
    ^             ~~~~~~~~~~~~~~
<source>:11:5: note: in instantiation of function template specialization
        'foo<char>' requested here
    foo('c');
    ^
<source>:6:29: note: expression evaluates to '1 == 8'
    static_assert(sizeof(T) == 8, "All types must have size 8");
                  ~~~~~~~~~~^~~~
```

In this case, there are two additional useful pieces of information, neither of which I can provide in a string literal: what `T` is and what `sizeof(T)` is. In this case, all three compilers do tell me what `T` is (gcc and MSVC explicitly, clang in a way that you can figure out) and two of them also tell me what `sizeof(T)` is. So that's not too bad.

But consider this slight variation:

```
template <typename T>
void foo(T t) {
    using U = std::remove_pointer_t<T>;
    static_assert(sizeof(U) == 8, "All types must have size 8");
```

```
    // ...
}
```

Think of `std::remove_pointer_t<T>` as representative of any kind of transformation. With this change, now only clang tells me that `U=char`.

That's good of clang and gcc to provide this extra information, but there's only so much that we can rely on compilers for. Generally speaking, at the point of assertion, the programmer writing it is going to have a better sense of what's useful than the compiler author - who needs to come up with something general purpose that works well in all cases. That's a tough line to walk - printing all the information that's useful without printing so much information that it's impossible to actually find the useful bits.

The compilers are doing better with every release, but in specific situations, the programmer will know what's important and can provide more and better information. If only they had any ability to do so.

§ 2.2 Forced constant evaluation failures

Consider the example:

```
auto f() -> std::string {
    return std::format("{} {:d}", 5, "not a number");
}
```

One of the cool things about `std::format` is that the format string is checked at compile time. The above is ill-formed: because `d` is not a valid format specifier for `const char*`. What is the compiler error that you get here?

— MSVC

```
<source>(6): error C7595: 'fmt::v9::basic_format_string<char,int,const char (&)
        [13]>::basic_format_string': call to immediate function is not a constant
        expression
C:\data\libraries\installed\x64-windows\include\fmt\core.h(2839): note: failure
        was caused by call of undefined function or one not declared 'constexpr'
C:\data\libraries\installed\x64-windows\include\fmt\core.h(2839): note: see usage
        of 'fmt::v9::detail::error_handler::on_error'
```

— GCC

```
/opt/compiler-explorer/gcc-trunk-20230108/include/c++/13.0.0/format: In function
        'std::string f()':
<source>:6:23:   in 'constexpr' expansion of 'std::basic_format_string<char, int,
        const char (&)[13]>("{} {:d}")'
/opt/compiler-explorer/gcc-trunk-20230108/include/c++/13.0.0/format:3634:19:   in
        'constexpr' expansion of '__scanner.std::__format::_Checking_scanner<char,
        int, char [13]>::<anonymous>.std::__format::_Scanner<char>::_M_scan()'
/opt/compiler-explorer/gcc-trunk-20230108/include/c++/13.0.0/format:3448:30:   in
        'constexpr' expansion of '((std::__format::_Scanner<char>*)this)-
        >std::__format::_Scanner<char>::_M_on_replacement_field()'
/opt/compiler-explorer/gcc-trunk-20230108/include/c++/13.0.0/format:3500:15:   in
        'constexpr' expansion of '((std::__format::_Scanner<char>*)this)-
        >std::__format::_Scanner<char>::_M_format_arg(__id)'
```

```
/opt/compiler-explorer/gcc-trunk-20230108/include/c++/13.0.0/format:3572:33:   in
        'constexpr' expansion of '((std::__format::_Checking_scanner<char, int,
        char [13]>*)this)->std::__format::_Checking_scanner<char, int, char
        [13]>::_M_parse_format_spec<int, char [13]>(__id)'
/opt/compiler-explorer/gcc-trunk-20230108/include/c++/13.0.0/format:3589:36:   in
        'constexpr' expansion of '((std::__format::_Checking_scanner<char, int,
        char [13]>*)this)->std::__format::_Checking_scanner<char, int, char
        [13]>::_M_parse_format_spec<char [13]>((__id - 1))'
/opt/compiler-explorer/gcc-trunk-20230108/include/c++/13.0.0/format:3586:40:   in
        'constexpr' expansion of '__f.std::formatter<char [13],
        char>::parse(((std::__format::_Checking_scanner<char, int, char
        [13]>*)this)->std::__format::_Checking_scanner<char, int, char [13]>::
        <anonymous>.std::__format::_Scanner<char>::_M_pc)'
/opt/compiler-explorer/gcc-trunk-20230108/include/c++/13.0.0/format:1859:26:   in
        'constexpr' expansion of '((std::formatter<char [13], char>*)this)-
        >std::formatter<char [13],
        char>::_M_f.std::__format::__formatter_str<char>::parse((* & __pc))'
/opt/compiler-explorer/gcc-trunk-20230108/include/c++/13.0.0/format:823:48: error:
        call to non-'constexpr' function 'void
        std::__format::__failed_to_parse_format_spec()'
  823 |         __format::__failed_to_parse_format_spec();
      |         ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~^~
/opt/compiler-explorer/gcc-trunk-20230108/include/c++/13.0.0/format:185:3: note:
        'void std::__format::__failed_to_parse_format_spec()' declared here
  185 |   __failed_to_parse_format_spec()
      |   ^~~~~~~~~~~~~~~~~~~~~~~~~~~~~
```

— Clang (with libstdc++, libc++ doesn't implement <format> yet):

```
<source>:6:24: error: call to consteval function 'std::basic_format_string<char,
        int, const char (&)[13]>::basic_format_string<char[8]>' is not a constant
        expression
    return std::format("{} {:d}", 5, "not a number");
                       ^
/opt/compiler-explorer/gcc-snapshot/lib/gcc/x86_64-linux-
        gnu/13.0.0/../../../../include/c++/13.0.0/format:823:2: note: non-
        constexpr function '__failed_to_parse_format_spec' cannot be used in a
        constant expression
        __format::__failed_to_parse_format_spec();
        ^
/opt/compiler-explorer/gcc-snapshot/lib/gcc/x86_64-linux-
        gnu/13.0.0/../../../../include/c++/13.0.0/format:1859:21: note: in call to
        '&__f._M_f->parse(__scanner._Scanner::_M_pc)'
      { return _M_f.parse(__pc); }
                    ^
/opt/compiler-explorer/gcc-snapshot/lib/gcc/x86_64-linux-
        gnu/13.0.0/../../../../include/c++/13.0.0/format:3586:35: note: in call to
        '&__f->parse(__scanner._Scanner::_M_pc)'
             this->_M_pc.advance_to(__f.parse(this->_M_pc));
                                              ^
/opt/compiler-explorer/gcc-snapshot/lib/gcc/x86_64-linux-
        gnu/13.0.0/../../../../include/c++/13.0.0/format:3589:6: note: in call to
        '&__scanner->_M_parse_format_spec(0)'
           _M_parse_format_spec<_Tail...>(__id - 1);
           ^
/opt/compiler-explorer/gcc-snapshot/lib/gcc/x86_64-linux-
        gnu/13.0.0/../../../../include/c++/13.0.0/format:3572:3: note: in call to
        '&__scanner->_M_parse_format_spec(1)'
```

```
                _M_parse_format_spec<_Args...>(__id);
                ^
/opt/compiler-explorer/gcc-snapshot/lib/gcc/x86_64-linux-
        gnu/13.0.0/../../../../include/c++/13.0.0/format:3500:2: note: in call to
        '&__scanner->_M_format_arg(1)'
        _M_format_arg(__id);
        ^
/opt/compiler-explorer/gcc-snapshot/lib/gcc/x86_64-linux-
        gnu/13.0.0/../../../../include/c++/13.0.0/format:3448:7: note: in call to
        '&__scanner->_M_on_replacement_field()'
                    _M_on_replacement_field();
                    ^
/opt/compiler-explorer/gcc-snapshot/lib/gcc/x86_64-linux-
        gnu/13.0.0/../../../../include/c++/13.0.0/format:3634:12: note: in call to
        '&__scanner->_M_scan()'
        __scanner._M_scan();
                  ^
<source>:6:24: note: in call to 'basic_format_string("{} {:d}")'
    return std::format("{} {:d}", 5, "not a number");
                       ^
/opt/compiler-explorer/gcc-snapshot/lib/gcc/x86_64-linux-
        gnu/13.0.0/../../../../include/c++/13.0.0/format:185:3: note: declared
        here
  __failed_to_parse_format_spec()
  ^
```

— GCC, using `{fmt}` trunk instead of libstdc++:

```
/opt/compiler-explorer/libs/fmt/trunk/include/fmt/core.h: In function 'std::string
        f()':
<source>:6:23:   in 'constexpr' expansion of 'fmt::v9::basic_format_string<char,
        int, const char (&)[13]>("{} {:d}")'
/opt/compiler-explorer/libs/fmt/trunk/include/fmt/core.h:2847:40:   in 'constexpr'
        expansion of 'fmt::v9::detail::parse_format_string<true, char,
        format_string_checker<char, int, char [13]> >
        (((fmt::v9::basic_format_string<char, int, const char (&)[13]>*)this)-
        >fmt::v9::basic_format_string<char, int, const char (&)[13]>::str_,
        fmt::v9::detail::format_string_checker<char, int, char [13]>
        (fmt::v9::basic_string_view<char>(((const char*)s))))'
/opt/compiler-explorer/libs/fmt/trunk/include/fmt/core.h:2583:44:   in 'constexpr'
        expansion of 'fmt::v9::detail::parse_replacement_field<char,
        format_string_checker<char, int, char [13]>&>((p + -1), end, (* &
        handler))'
/opt/compiler-explorer/libs/fmt/trunk/include/fmt/core.h:2558:38:   in 'constexpr'
        expansion of '(& handler)->fmt::v9::detail::format_string_checker<char, int,
        char
        [13]>::on_format_specs(adapter.fmt::v9::detail::parse_replacement_field<char,
        format_string_checker<char, int, char [13]>&>(const char*, const char*,
        format_string_checker<char, int, char [13]>&)::id_adapter::arg_id, (begin +
        1), end)'
/opt/compiler-explorer/libs/fmt/trunk/include/fmt/core.h:2727:51:   in 'constexpr'
        expansion of '((fmt::v9::detail::format_string_checker<char, int, char
        [13]>*)this)->fmt::v9::detail::format_string_checker<char, int, char
        [13]>::parse_funcs_[id](((fmt::v9::detail::format_string_checker<char,
        int, char [13]>*)this)->fmt::v9::detail::format_string_checker<char, int,
        char [13]>::context_)'
```

```
/opt/compiler-explorer/libs/fmt/trunk/include/fmt/core.h:2641:17:   in 'constexpr'
        expansion of 'f.fmt::v9::formatter<const char*, char,
        void>::parse<fmt::v9::detail::compile_parse_context<char> >((* & ctx))'
/opt/compiler-explorer/libs/fmt/trunk/include/fmt/core.h:2784:35:   in 'constexpr'
        expansion of 'fmt::v9::detail::parse_format_specs<char>((& ctx)-
        >fmt::v9::detail::compile_parse_context<char>::
        <anonymous>.fmt::v9::basic_format_parse_context<char>::begin(), (& ctx)-
        >fmt::v9::detail::compile_parse_context<char>::
        <anonymous>.fmt::v9::basic_format_parse_context<char>::end(),
        ((fmt::v9::formatter<const char*, char, void>*)this)-
        >fmt::v9::formatter<const char*, char, void>::specs_,
        ctx.fmt::v9::detail::compile_parse_context<char>::<anonymous>, type)'
/opt/compiler-explorer/libs/fmt/trunk/include/fmt/core.h:2468:37:   in 'constexpr'
        expansion of
        'parse_presentation_type.fmt::v9::detail::parse_format_specs<char>(const
        char*, const char*, dynamic_format_specs<>&,
        fmt::v9::basic_format_parse_context<char>&, type)::<unnamed
        struct>::operator()(fmt::v9::presentation_type::dec, ((int)integral_set))'
/opt/compiler-explorer/libs/fmt/trunk/include/fmt/core.h:2395:49: error: call to
        non-'constexpr' function 'void fmt::v9::detail::throw_format_error(const
        char*)'
 2395 |       if (!in(arg_type, set)) throw_format_error("invalid format
        specifier");
      |
        ~~~~~~~~~~~~~~~~~~^~~~~~~~~~~~~~~~~~~~~~~~~~~
/opt/compiler-explorer/libs/fmt/trunk/include/fmt/core.h:646:27: note: 'void
        fmt::v9::detail::throw_format_error(const char*)' declared here
  646 | FMT_NORETURN FMT_API void throw_format_error(const char* message);
      |                           ^~~~~~~~~~~~~~~~~~
```

All the compilers reject the code, which is good. MSVC gives you no information at all. Clang indicates that there's something wrong with some format spec, but doesn't show enough information to know what types are involved (is it the `5` or the `"not a number"`?). GCC does the best in that you can actually tell that the problem argument is a `char[13]` (if you really carefully peruse the compile error), but otherwise all you know is that there's *something* wrong with the format spec.

This isn't a standard library implementation problem - the error gcc gives when using `{fmt}` isn't any better. If you carefully browse the message, you can see that it's the `const char*` specifier that's the problem, but otherwise all you know is that it's invalid.

The problem here is that the only way to "fail" here is to do something that isn't valid during constant evaluation time, like throw an exception or invoke an undefined function. And there's only so much information you can provide that way. You can't provide the format string, you can't point to the offending character.

Imagine how much easier this would be for the end-user to determine the problem and then fix if the compiler error you got was something like this:

```
format("{} {:d}", int, const char*)
               ^         ^
'd' is an invalid type specifier for arguments of type 'const char*'
```

That message might not be perfect, but it's overwhelmingly better than anything that's possible today. So we should at least make it possible tomorrow.

2.3 General compile-time debugging

The above two sections were about the desire to emit a compile error, with a rich diagnostic message. But sometimes we don't want to emit an *error*, we just want to emit *some information*.

When it comes to runtime programming, there are several mechanisms we have for debugging code. For instance, you could use a debugger to step through it or you could litter your code with print statements. When it comes to *compile-time* programmer, neither option is available. But it would be incredibly useful to be able to litter our code with *compile-time* print statements. This was the initial selling point of Circle: want compile-time prints? That's just `@meta printf`.

There's simply no way I'm aware of today to emit messages at compile-time other than forcing a compile error, and even those (as hinted at above) are highly limited.

2.4 Prior Work

[N4433] previously proposed extending `static_assert` to support arbitrary constant expressions. That paper was discussed in Lenexa in 2015. The minutes indicate that that there was concern about simply being able to implement a useful `format` in `constexpr` (`{fmt}` was just v1.1.0 at the time). Nevertheless, the paper was well received, with a vote of 12-3-9-1-0 to continue work on the proposal. Today, we know we can implement a useful `format` in `constexpr`. We already have it!

[P0596R1] previously proposed adding `std::constexpr_trace` and `std::constexpr_assert` facilities - the former as a useful compile-time print and the latter as a useful compile-time assertion to emit a useful message. That paper was discussed in Belfast in 2019, where these two facilities were very popular (16-8-1-0-0 for compile-time print and 6-14-2-0-0 for compile-time assertion). The rest of the discussion was about broader compilation models that isn't strictly related to these two.

In short, the kind of facility I'm reviving here were already previously discussed and received *extremely favorably*. 15-1, 24-0, and 20-0. It's just that then the papers disappeared, so I'm bringing them back.

3 To `std::format` or not to `std::format`?

That is the question. Basically, when it comes to emitting some kind of text (via whichever mechanism - whether `static_assert` or a compile-time print or a compile-time error), we have to decide whether or not to bake `std::format` into the API. The advantage of doing so would be ergonomics, the disadvantage would be that it's a complex library to potential bake into the language - and some people might want these facilities in a context where they're not using `std::format`, for hwatever reason.

But there's also a bigger issue: while I said above that we have a useful `format` in `constexpr`, that wasn't *entirely* accurate. The parsing logic is completely `constexpr` (to great effect), but the formatting logic currently is not. Neither `std::format` nor `fmt::format` are declared `constexpr` today. In order to be able to even consider the question of using `std::format` for generating compile-time strings, we have to first ask to what extent this is even feasible.

I think there are currently two limitations (excluding just adding `constexpr` everywhere and possibly dealing with some algorithms that happen to not be `constexpr`-friendly):

1. formatting floating-point types is not possible right now (we made the integral part of `std::to_chars() constexpr` [P2291R3], but not the floating point).
2. `fmt::format` and `std::format` rely on type erasing user-defined types, and that's currently impossible to do at `constexpr` time.

I am not in a position to say how hard the first of the two is (it's probably pretty hard?), but I have a separate paper addressing the second [P2747R0]. If we can do any kind of type erasure at all, then it's probably not too much work to get the rest of format working - even if we ignore floating point entirely. Without compile-time type erasure, it's still possible to write just a completely different consteval formatting API - but I doubt people would be too happy about having to redo all that work.

We will eventually have `constexpr std::format`, I'm just hoping that we can do so with as little overhead on the library implementation itself (in terms of lines of code) as possible.

§ 4 Improving `static_assert`

There are basically two approaches we can take to generalizing the kinds of output `static_assert` can produce.

We can allow any constant expression that is some kind of range of `char`, `wchar_t`, `char8_t`, `char16_t`, or `char32_t`. Like this one:

```
static_assert(cond, std::format("The value is {}", 42));
```

Or we can embed `std::format` into the declaration itself:

```
static_assert(cond, "The value is {}", 42);
```

The latter is definitely more ergonomic than the former, but only because you don't have to write the call to `std::format`. However, it has a couple problems.

One big problem is: what does this mean?

```
static_assert(cond, "T{} must be a valid expression.");
```

Today, that's a valid declaration. But that isn't a valid format string without format arguments - you'd have to escape the braces to get the same behavior. We could resolve this issue by saying that:

— `static_assert(cond, `*`string-literal`*`)` is the existing behavior

— `static_assert(cond, `*`string-literal, expression-list...`*`)` where *`expression-list`* contains at least one expression is the new behavior

This seems… not great. Rust went this approach before and recently changed it in the 2021 edition. From panic macro consistency:

> The `panic!()` macro is one of Rust's most well known macros. However, it has some subtle surprises that we can't just change due to backwards compatibility.
>
> ```
> // Rust 2018
> panic!("{}", 1); // Ok, panics with the message "1"
> panic!("{}"); // Ok, panics with the message "{}"
> ```
>
> […]
>
> This will especially be a problem once implicit format arguments are stabilized. That feature will make `println!("hello {name}")` a short-hand for `println!("hello {}", name)`. However, `panic!("hello {name}")` would not work as expected, since `panic!()` doesn't process a single argument as format string.
>
> To avoid that confusing situation, Rust 2021 features a more consistent `panic!()` macro. The new `panic!()` macro will no longer accept arbitrary expressions as the only argument. It will, just like `println!()`, always process the first argument as format string.

Generally speaking, probably not a great idea to adopt a language design that another language is explicitly moving away from.

And we, unfortunately, also can't simply change the meaning of existing `static_assert` declarations in order to start treating them uniformly as format strings. If we had some mechanism to break source compatibility in an opt-in way (like, say, Rust editions), then that's something we could consider. But we don't, so we can't.

This approach has another problem, which is tying in a complex library mechanism into a language feature. But I'm not really sure it's worth dwelling on in light of the fact that we can't really go this route anyway.

That leaves the other idea - requiring the user to write `std::format` themselves:

```
static_assert(cond, std::format("The value is {}", 42));
```

Tedious, but at least it makes a facility possible that currently is not. In order to specify this, we'd need to extend the definition of `static_assert`. Which [P2741R3] has already done for us.

§ 5 Improving compile-time diagnostics

While in `static_assert`, I don't think we can have a `std::format()`-based API, for compile-time diagnostics, I think we should. In particular, the user-facing API should probably be something like this:

```
namespace std {
  template<class... Args>
    constexpr void constexpr_print(format_string<Args...> fmt, Args&&...
        args);
  template<class... Args>
```

```
    constexpr void constexpr_error(format_string<Args...> fmt, Args&&...
        args);
}
```

But we'll probably still need a lower-level API as well. Something these facilities can be implemented on top of, that we might want to expose to users anyway in case they want to use something other than `std::format` for their formatting needs. Perhaps something like this:

```
namespace std {
  constexpr void constexpr_print_str(string_view);
  constexpr void constexpr_error_str(string_view);
}
```

That is really the minimum necessary, and the nice `format` APIs can then trivially be implemented by invoking `std::format` and then passing in the resulting `std::string`.

But in order to talk about what these APIs actually do and what their effects are, we need to talk about a fairly complex concept: predictability.

§ 5.1 Predictability

[P0596R1] talks about predictability introducing this example:

```
template<typename> constexpr int g() {
    std::__report_constexpr_value("in g()\n");
    return 42;
}

template<typename T> int f(T(*)[g<T>()]); // (1)
template<typename T> int f(T*);           // (2)
int r = f<void>(nullptr);
```

When the compiler resolves the call to `f` in this example, it substitutes `void` for `T` in both declarations (1) and (2). However, for declaration (1), it is unspecified whether `g<void>()` will be invoked: The compiler may decide to abandon the substitution as soon as it sees an attempt to create "an array of void" (in which case the call to `g<void>` is not evaluated), or it may decide to finish parsing the array declarator and evaluate the call to `g<void>` as part of that.

We can think of a few realistic ways to address/mitigate this issue:

1. Make attempts to trigger side-effects in expressions that are "tentatively evaluated" (such as the ones happening during deduction) ill-formed with no diagnostic required (because we cannot really require compilers to re-architect their deduction system to ensure that the side-effect trigger is reached).
2. Make attempts to trigger side-effects in expressions that are "tentatively evaluated" cause the expression to be non-constant. With our example that would mean that even a call `f<int>(nullptr)` would find (1) to be nonviable because `g<int>()` doesn't produce a constant in that context.

> 3. Introduce a new special function to let the programmer control whether the side effect takes place anyway. E.g., `std::is_tentatively_constant_evaluated()`. The specification work for this is probably nontrivial and it would leave it unspecified whether the call to `g<void>` is evaluated in our example.
>
> We propose to follow option 2. Option 3 remains a possible evolution path in that case, but we prefer to avoid the resulting subtleties if we can get away with it.

As well as:

> There is another form of "tentative evaluation" that is worth noting. Consider:
>
> ```
> constexpr int g() {
>     std::__report_constexpr_value("in g()\n");
>     return 41;
> }
> int i = 1;
> constexpr int h(int p) {
>     return p == 0 ? i : 1;
> }
> int r = g()+h(0); // Not manifestly constant-evaluated but
>                   // g() is typically tentatively evaluated.
> int s = g()+1;    // To be discussed.
> ```
>
> Here `g()+h(0)` is not a constant expression because `i` cannot be evaluated at compile time. However, the compiler performs a "trial evaluation" of that expression to discover that. In order to comply with the specification that `__report_constexpr_value` only produce the side effect if invoked as part of a "manifestly constant-evaluated expression", two implementation strategies are natural:
>
> 1. "Buffer" the side effects during the trial evaluation and "commit" them only if that evaluation succeeds.
> 2. Disable the side effects during the trial evaluation and repeat the evaluation with side effects enabled if the trial evaluation succeeds.
>
> The second option is only viable because "output" as a side effect cannot be observed by the trial evaluation. However, further on we will consider another class of side effects that can be observed within the same evaluation that triggers them, and thus we do not consider option 2 a viable general implementation strategy.
>
> The first option is more generally applicable, but it may impose a significant toll on performance if the amount of side effects that have to be "buffered" for a later "commit" is significant.
>
> An alternative, therefore, might be to also consider the context of a non-constexpr variable initialization to be "tentatively evaluated" and deem side-effects to be non-constant in that case (i.e., the same as proposed for evaluations during deduction). In the example above, that means that `g()+1` would not be a constant expression either (due to the potential side effect by

> `__report_constexpr_value` in an initializer that is allowed to be non-constant) and thus `s` would not be statically initialized.

Now, my guiding principle here is that if we take some code that currently works and does some constant evaluation, and add to that code a `constexpr_print` statement, the *only* change in behavior should be the addition of output during compile time. For instance:

```
constexpr auto f(int i) -> int {
    std::constexpr_print("Called f({})\n", i);
    return i;
}

int x = f(2);
```

WIthout the `constepr_print`, this variable is constant-initialized. WIth it, it should be also. It would be easier to deal with the language if we didn't have all of these weird rules. For instance, if you want constant initialize, use `constinit`, if you don't, there's no tentative evaluation. But we can't change that, so this is the language we have.

I think buffer-then-commit is right approach. But also for the first example, that tentative evaluation in a manifestly constant evaluated context is *still* manifestly constant evaluated. It's just unspecified whether the call happens. That is: in the first example, the call `f<void>(nullptr)` may or may not print `"in g()\n"`. It's unspecified. It may make constexpr output not completely portable, but I don't think any of the alternatives are palatable.

§ 5.2 Predictability of Errors

An interesting follow-on is what happens here:

```
constexpr auto f(int i) -> int {
    if (i < 0) {
        std::constexpr_error_str("cannot invoke f with a negative number");
    }
    return i;
}

constexpr int a = f(-1);
int b = f(-1);
```

Basically the question is: what are the actual semantics of `constexpr_error`?

If we just say that evaluation (if manifestly constant-evaluated) causes the evaluation to not be a constant, then `a` is ill-formed but `b` would be (dynamically) initialized with `-1`.

That seems undesirable: this is, after all, an error that we have the opportunity to catch. This is the only such case: all other manifestly constant evaluated contexts don't have this kind of fall-back to runtime. So I think it's not enough to say that constant evaluation fails, but rather that the entire program is ill-formed in this circumstance: both `a` and `b` are ill-formed.

We also have to consider the predictability question for error-handling. Here's that same example again:

```
template<typename> constexpr int g(int i) {
    if (i < 0) {
        std::constexpr_error_str("can't call g with a negative number");
    }
    return 42;
}

template<typename T> int f(T(*)[g<T>(-1)]); // (1)
template<typename T> int f(T*);             // (2)
int r = f<void>(nullptr);
```

If `g<T>(-1)` is called, then it'll hit the `constexpr_error_str` call. But it might not be called. I think saying that if it's called, then the program is ill-formed, is probably fine. If necessary, we can further tighten the rules for substitution and actually specify one way or another (actually specify that `g` is *not* invoked because by the time we lexically get there we know that this whole type is ill-formed, or specify that `g` *is* invoked because we atomically substitute one type at a time), but it's probably not worth the effort.

Additionally, we could take a leaf out of the book of speculative evaluation. I think of the tentative evaluation of `g<T>(-1)` is this second example *quite* differently from the tentative *constant* evaluation of `f(-1)` in the first example. `f` is *always* evaluated, it's just that we have this language hack that it ends up potentially being evaluated two different ways. `g` isn't *necessarily* evaluated. So there is room to treat these different. If `g` is tentatively evaluated, then we buffer up our prints and errors - such that if it eventually *is* evaluated (that overload is selected), we then emit all the prints and errors. Otherwise, there is no output. That is, we specify *no* output if the function isn't selected. Because the evaluation model is different here - that `f` is always constant-evaluated initially - I don't think of these as inconsistent decisions.

§ 5.3 Error-Handling in General

Basically, in all contexts, you probably wouldn't want to *just* `std::constexpr_error`. Well, in a `consteval` function, that's all you'd have to do. But in a `constexpr` function that might be evaluated at runtime, you probably still want to fail.

But the question is, *how* do you want to fail? There are so many different ways of failing

- throw an exception (of which kind?)
- return some kind of error object (return code, `unexpected`, `false`, etc.)
- `std::abort()`
- `std::terminate()`
- etc.

Which fallback depends entirely on the circumstance. For `formatter<T>::parse`, one of my motivating examples here, we have to throw a `std::format_error` in this situation. The right pattern there would probably be:

```
if consteval {
    std::constexpr_error("Bad specifier {}", *it);
} else {
    throw std::format_error(std::format("Bad specifier {}", *it));
}
```

Which can be easily handled in its own API:

```
template <typename... Args>
constexpr void format_parse_failure(format_string<Args...> fmt, Args&&...
        args) {
    if consteval {
        constexpr_error(fmt, args...);
    } else {
        throw format_error(format(fmt, args...));
    }
}
```

So we should probably provide that as well (under whichever name).

But that's a format-specific solution. But a similar pattern works just fine for other error handling mechanisms, except for wanting to return an object (unless your return object happens to have a string part - since the two cases end up being very dfferent). I think that's okay though - at least we have the utility.

§ 5.4 Errors in Constraints

Let's take a look again at the example I showed earlier:

```
constexpr auto f(int i) -> int {
    if (i < 0) {
        std::constexpr_error_str("cannot invoke f with a negative number");
    }
    return i;
}

template <int I> requires (f(I) % 2 == 0)
auto g() -> void;
```

Here, `g<2>()` is obviously fine and `g<3>()` will not satisfy the constraints as usual, nothing interesting to say about either call. But what about if we try `g<-1>()`? Based on our currently language rules and what's being proposed here, `f(-1)` is not a constant expression, and the rule we have in 13.5.2.3 [temp.constr.atomic]/3 is:

> If substitution results in an invalid type or expression, the constraint is not satisfied. Otherwise, the lvalue-to-rvalue conversion is performed if necessary, and E shall be a constant expression of type `bool`.

That is, `g<-1>()` is ill-formed, with our current rules. But loosening those rules to allow non-constants (still of type `bool`) would allow for more useful constructs like this. After all, the above is pretty conceptually similar to having written:

```
constexpr auto f(int i) -> std::optional<int> {
    if (i < 0) {
        return std::nullopt;
    }
    return i;
}

template <int I> requires (f(I) % 2 == 0)
auto g() -> void;
```

And here, `g<-1>()` would be a substitute failure. So why not in the original? This question also ends up being closely tied into what it would mean to have constexpr exceptions.

6 Proposal

This paper proposes the following:

1. Introduce a new compile-time diagnostic API that only has effect if manifestly constant evaluated: `std::constexpr_print_str(msg)`

2. Introduce two new compile-time error APIs, that only has effect if manifestly constant evaluated: `std::constexpr_error_str(msg)` and `std::constexpr_fail_str(msg)` will both cause the program to be ill-formed. The second additionally causes the evaluation to not be a constant expression. EWG took a poll in February to encourage work on the ability to print multiple errors per constant evaluation but still result in a failed TU:

SF	F	N	A	SA
5	10	3	1	0

   This implies having a function that can make the program ill-formed, and a stronger one that also makes the constant evaluation fail to be a constant expression (to ensure that the next statement is not executed).

3. Pursue `constexpr std::format(fmt_str, args...)`, which would then allow us to extend the above API with:

   a. `std::constexpr_print(fmt_str, args...)`

   b. `std::constexpr_error(fmt_str, args...)` and `std::constexpr_fail(fmt_str, args...)`

   c. a `format`-specific helper `std::format_parse_error(fmt_str, args...)` that either calls `std::constexpr_error` or throws a `std::format_error`, depending on context.

7 Wording

We don't quite have `constexpr std::format` yet (although with the addition of [P2738R1] we're probably nearly the whole way there), so the wording here only includes (1) and (2) above - with the

understanding that a separate paper will materialize to produce a `constexpr std::format` and then another separate paper will add `std::constexpr_print` and `std::constexpr_error` (the nicer names, with the more user-friendly semantics).

Add to 21.3.3 [meta.type.synop]:

```
// all freestanding
namespace std {
  // ...

  // [meta.const.eval], constant evaluation context
  constexpr bool is_constant_evaluated() noexcept;
  consteval bool is_within_lifetime(const auto*) noexcept;

+ constexpr void constexpr_print_str(string_view) noexcept;
+ constexpr void constexpr_error_str(string_view) noexcept;
+ constexpr void constexpr_fail_str(string_view) noexcept;

}
```

Add to 21.3.11 [meta.const.eval]:

```
constexpr void constexpr_print_str(string_view msg) noexcept;
```

6 *Effects*: During constant evaluation, a diagnostic message is issued including the text of `msg`. Otherwise, no effect.

```
constexpr void constexpr_error_str(string_view msg) noexcept;
```

7 *Effects*: During constant evaluation, the program is ill-formed and a diagnostic message is issued including the text of `msg`. Otherwise, no effect.

```
constexpr void constexpr_fail_str(string_view msg) noexcept;
```

8 *Effects*: During constant evaluation, the program is ill-formed, a diagnostic message is issued including the text of `msg`, and the evaluation of this call is not a *core constant expression* ([expr.const]). Otherwise, no effect.

§ 8 References

[N4433] Michael Price. 2015-04-09. Flexible static_assert messages.
https://wg21.link/n4433

[P0596R1] Daveed Vandevoorde. 2019-10-08. Side-effects in constant evaluation: Output and consteval variables.
https://wg21.link/p0596r1

[P2291R3] Daniil Goncharov, Karaev Alexander. 2021-09-23. Add Constexpr Modifiers to Functions to_chars and from_chars for Integral Types in Header.
https://wg21.link/p2291r3

[P2738R1] Corentin Jabot, David Ledger. 2023-02-13. constexpr cast from void*: towards constexpr type-erasure.
https://wg21.link/p2738r1

[P2741R0] Corentin Jabot. 2022-12-09. user-generated static_assert messages.
https://wg21.link/p2741r0

[P2741R3] Corentin Jabot. 2023-06-16. user-generated static_assert messages.
https://wg21.link/p2741r3

[P2747R0] Barry Revzin. 2022-12-16. Limited support for `constexpr void*`.
https://wg21.link/p2747r0

[P2758R0] Barry Revzin. 2023-01-13. Emitting messages at compile time.
https://wg21.link/p2758r0